Claude Usage Tracking & Conversation Caps

Two related features:

1. **Usage tracking** — every call to Anthropic's `messages.create` is logged to Supabase with token counts and the USD cost computed at insert time.
2. **Conversation caps** — per-session and per-phone limits short-circuit the Claude call (with a warm consultant-handoff reply) before runaway loops or abuse rack up spend.

---

1. Usage Tracking

Schema

supabase/migrations/20260428100000_claude_usage_tracking.sql

`claude_usage` — one row per `messages.create` call.

Column	Notes
`session_id`	FK to `sessions`, nullable (set null on session delete)
`phone_number`	denormalised for cheap per-phone queries
`role`	`'client'` / `'staff'` / `'unknown'`
`model`	e.g. `claude-opus-4-7`
`call_purpose`	`'main'` / `'tool_loop'` / `'intent_classify'`
`input_tokens`	
`output_tokens`	
`cache_creation_tokens`	5-min ephemeral cache writes
`cache_read_tokens`	cache hits — billed at ~10% of input rate
`cost_usd`	computed at insert from current pricing table
`created_at`	

Indexes: `session_id`, `phone_number`, `created_at`, `model`.

`claude_usage_daily` — view aggregating by day × phone × role × model. Use this for dashboards; querying the raw table for a month-long window will scan a lot of rows.

Pricing table

src/services/claudePricing.service.ts — USD per million tokens for Opus 4.7, Opus 4.6, Sonnet 4.6, Haiku 4.5. Includes the four tiers: input, output, cache-write (5-min), cache-read.

Important: the cost is computed **at insert time** and frozen on the row. When Anthropic changes prices, update the table — historical rows stay correct.

Unknown models fall back to Opus pricing (deliberately too-high) so a typo logs a visible cost rather than silently zeroing out.

Where logging fires

src/services/claude.service.ts — three call sites, all wrapped in the `logUsage()` helper near line 450:

Line	Call	`call_purpose`
~675	first/main turn	`main`
~1840	tool-loop iterations	`tool_loop`
~2150	intent classifier	`intent_classify`

Logging is fire-and-forget — a Supabase outage will not break a live conversation.

Cap counters on the session

The same `logClaudeUsage()` call also bumps `sessions.message_count` and `sessions.token_count` via the `increment_session_usage(p_session_id, p_tokens)` Postgres function. This avoids running `sum()` on `claude_usage` for every inbound message just to check the cap.

Useful queries

```sql
-- Today's spend
select sum(cost_usd) as usd, sum(call_count) as calls
from claude_usage_daily
where day = current_date;

-- Top 10 spenders this week
select phone_number, sum(cost_usd) as usd, sum(call_count) as calls
from claude_usage_daily
where day >= current_date - interval '7 days'
group by 1 order by usd desc limit 10;

-- Cost per call_purpose (find expensive paths)
select call_purpose, count(*) as calls, sum(cost_usd) as usd,
avg(input_tokens) as avg_in
from claude_usage
where created_at >= current_date - interval '7 days'
group by 1 order by usd desc;

-- Cache effectiveness
select
    sum(cache_read_tokens)::float / nullif(sum(input_tokens) +
sum(cache_read_tokens), 0) as cache_hit_ratio
from claude_usage
where created_at >= current_date - interval '24 hours';
```

Retention

The table is unbounded today. At a few hundred rows per day this is fine for months. When it grows, roll up older rows into `claude_usage_daily` and truncate `claude_usage` rows older than 90 days — the daily view keeps the aggregate history.

---

2. Conversation Caps

Constants

src/controllers/webhook.controller.ts:47-49:

```ts
const CAP_MESSAGES_PER_SESSION = 50;
const CAP_TOKENS_PER_SESSION = 200_000;
const CAP_MESSAGES_PER_DAY = 100;
```

Who they apply to

- **Clients, leads, unknown phones** — capped.
- **Staff** (`crmEntity.type === 'user'`) — exempt. Their tool-driven workflows legitimately rack up turns, and tool surface is already gated by role permissions.

When they fire

For each inbound message from a non-staff user, before the Claude call:

1. If `session.cap_blocked_at` is set → reply `CAP_BLOCKED_REPLY` (already-notified ack) and return. This skips even the daily-count query.
2. Else if session messages ≥ 50 OR session tokens ≥ 200k OR phone messages in last 24h ≥ 100 → reply `CAP_HIT_REPLY` (warm consultant handoff), set `cap_blocked_at` on the session, return.
3. Else proceed to Claude as normal.

The session stays cap-blocked until the 30-min inactivity timeout expires it (src/services/supabase.service.ts:6). After expiry the user gets a fresh session and the counters reset.

Reply copy

```
CAP_HIT_REPLY:
"We've covered a lot today! 🙂 To make sure you get the right help, I'll
loop in a TTT consultant — they'll be in touch shortly. You can message
again anytime."

CAP_BLOCKED_REPLY:
"Thanks — a TTT consultant has already been notified and will be in touch.
No need to reply here."
```

`CAP_HIT_REPLY` is the first-time hit (warm, sets expectations). `CAP_BLOCKED_REPLY` is for any further messages in the same session — terse, prevents the user thinking the bot is just ignoring them.

Tuning the thresholds

Current values are conservative guesses. After a week or two of `claude_usage` data:

```
-- 99th-percentile session size for clients — anything above this is
suspicious
select
    percentile_cont(0.95) within group (order by message_count) as p95_msgs,
    percentile_cont(0.99) within group (order by message_count) as p99_msgs,
    percentile_cont(0.95) within group (order by token_count)   as
p95_tokens,
    percentile_cont(0.99) within group (order by token_count)   as
p99_tokens
from sessions
where role_id is null  -- clients/leads/unknown only
  and created_at >= current_date - interval '14 days';
```

Set the cap at p99 + a margin. Don't set at p95 — that frustrates real heavy users.

What's not implemented (deliberate)

- **No human escalation hook on cap hit.** The reply *says* a consultant will be in touch but no consultant is automatically pinged. If you want that, wire `markSessionCapBlocked` to call `caseService.markEscalated` or send a Slack alert.
- **No staff cap.** Staff can in theory burn budget too. Add a staff-tier cap if logs show it.
- **No per-IP / per-WABA cap.** Meta's webhook doesn't expose IP, and abuse via a single phone number is the realistic threat model.

Manual testing

To force a cap on a test phone without sending 50 messages:

```
update sessions
   set message_count = 60
 where phone_number = '<test_phone>'
   and last_active > now() - interval '30 minutes';
```

Send one more message — should get `CAP_HIT_REPLY` and the session should now have `cap_blocked_at`. Send another — should get `CAP_BLOCKED_REPLY`. Wait 30 min for session expiry, send again — should be a fresh session, normal reply.